



Università
degli Studi
di Palermo



Graph Neural Networks

INTELLIGENZA ARTIFICIALE PER L'INGEGNERIA BIOMEDICA (EX ROBOTICA MEDICA)
a.a. 2025/2026

Prof. Roberto Pirrone



Sommario

- Graph Neural Networks
 - Graph NN
 - Graph pre-processing
 - Applicazioni GNN nel task

Generalità

«Perché il Deep Learning sui Grafi?»

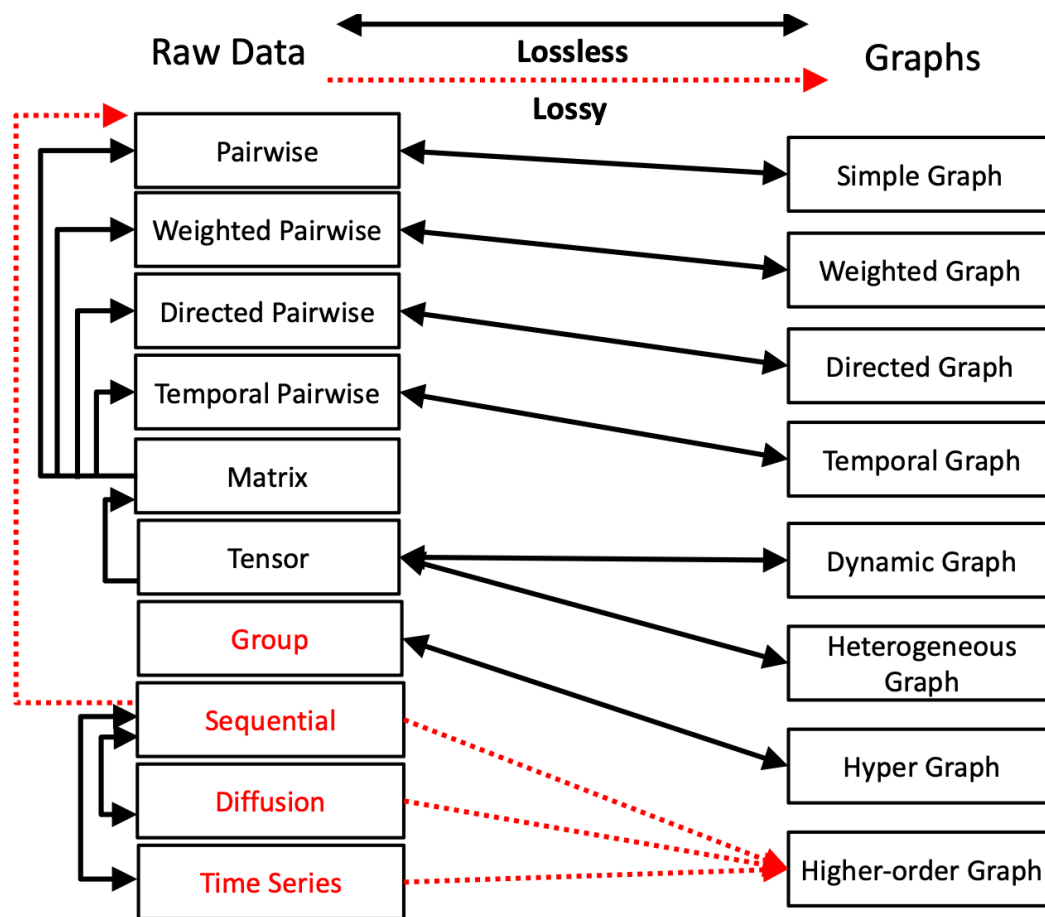
Principalmente per 2 ragioni

1. I grafi forniscono una rappresentazione universale del dato

- Social Networks
- Networks dei trasporti
- Protein-Protein interaction
- Knowledge graphs
- Brain networks

Generalità

Molti dati possono essere trasformati in grafi



Generalità

«Perché il Deep Learning sui Grafi?»

Principalmente per 2 ragioni

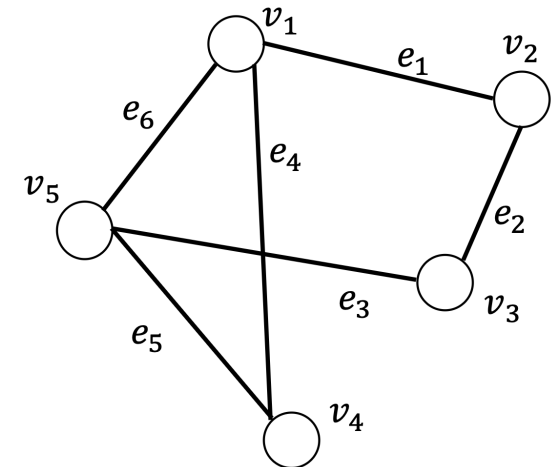
2. Molti problemi real-worlds possono essere affrontati come un piccolo set di operazioni sui grafi
 - Inferire attributi sui nodi
 - Individuare nodi anomali (e.g. nelle comunicazioni troviamo spammers o terroristi)
 - Identificare geni implicati nell'insorgenza di patologie
 - Drug-Target interaction
 - Predizione degli effetti collaterali dei farmaci

Generalità

«Cos'è un grafo?»

- Un grafo può essere definito come $G = \{V, E\}$

- $V = \{v_1, v_2, \dots, v_N\}$ un set di $N = |V|$ **nodi**
 - *Grafi sociali gli utenti sono visti come nodi*
 - *Nelle molecole saranno gli atomi visti come nodi*
- $E = \{e_1, e_2, \dots, e_M\}$ un set di M **archi**
 - *Descrivono la connessione tra i nodi*



***Ogni grafo può essere rappresentato attraverso
una matrice di adiacenza***



Università
degli Studi
di Palermo



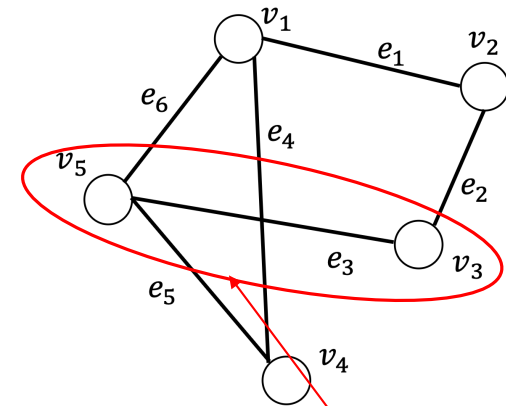
dipartimento
di ingegneria
unipa



Generalità

Matrice di adiacenza

- Per un grafo $G = \{V, E\}$ la sua matrice di adiacenza è descritta come $A \in \{0, 1\}^{N \times N}$
- L'elemento all'interno $A_{i,j}$ della matrice di adiacenza rappresenta la connettività tra i nodi v_i e v_j



$$A = \begin{pmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

Generalità

Grafi eterogenei

- Un grafo può essere definito come $G = \{V, E\}$

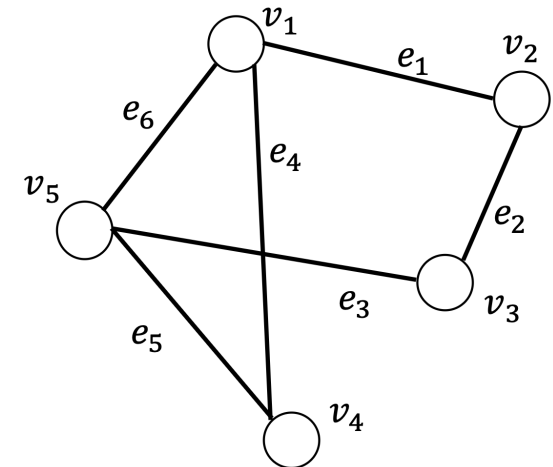
- $V = \{v_1, v_2, \dots, v_N\}$ un set di $N = |V|$ **nodi**

- *Grafi sociali gli utenti sono visti come nodi*
- *Nelle molecole saranno gli atomi visti come nodi*

- $E = \{e_1, e_2, \dots, e_M\}$ un set di M **archi**

- *Descrivono la connessione tra i nodi*

- Nodi ed archi vengono assegnati ad un tipo T attraverso una funzione di mapping $\phi_n: V \rightarrow T_n$ e $\phi_m: E \rightarrow T_m$

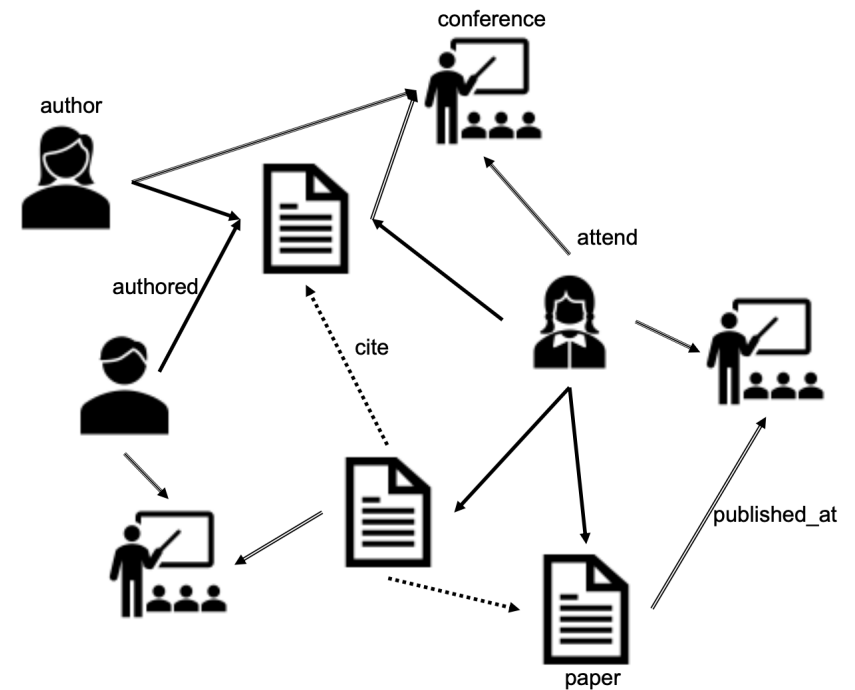


Generalità

Grafi eterogenei

Grafo universitario:

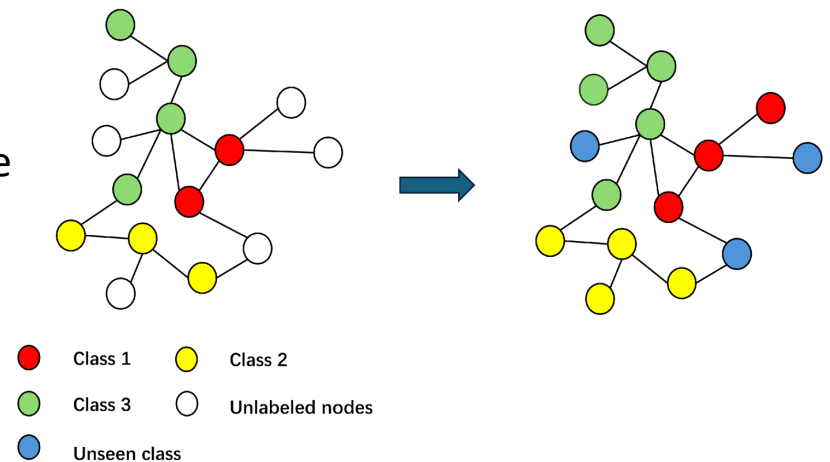
- 3 tipi di nodi:
 - Autori
 - Paper
 - Eventi
- Gli archi descrivono le pubblicazioni e le citazioni



Task di analisi dei grafi

- **Node Classification**

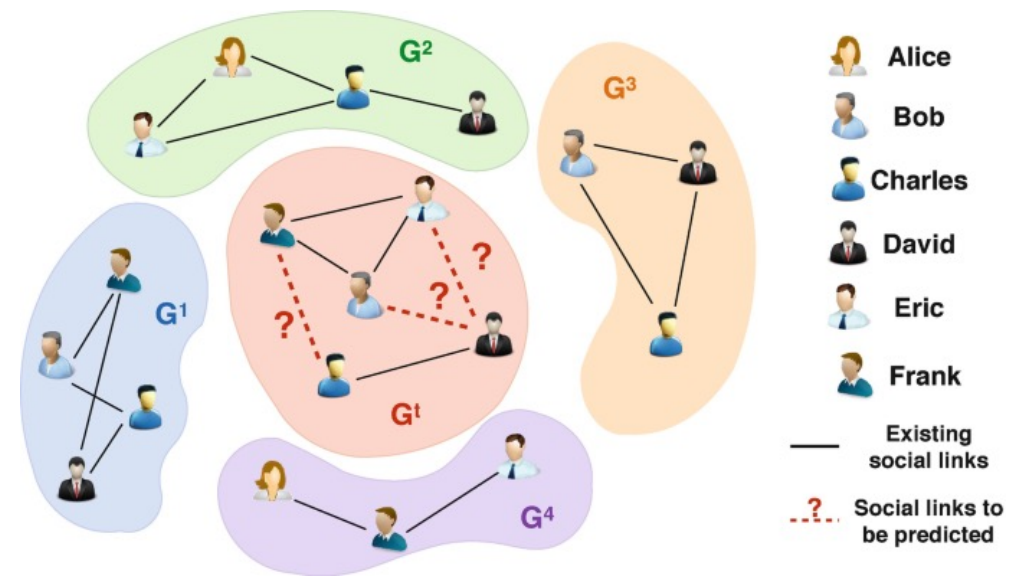
- I nodi contengono molte feature che spesso possono essere utilizzate come etichette
- Nei grafi di social networks i nodi sono gli utenti e le feature sono:
 - L'età
 - Informazioni demografiche
 - Genere
 - Occupazione
- Sulla base di queste feature possiamo classificare i singoli nodi



Task di analisi dei grafi

- **Link Prediction**

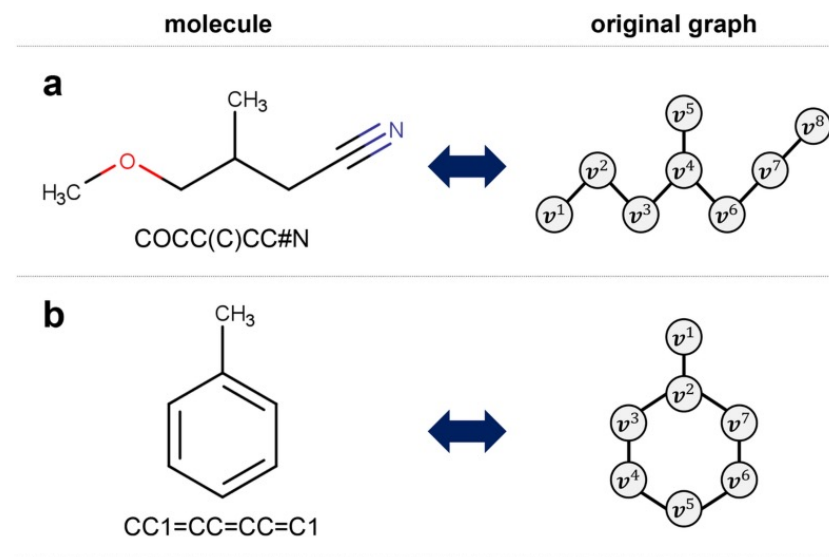
- Molti grafi non hanno tutte le interazioni tra nodi esplicitate e possono essere predette
- I settori che possono beneficiare di applicazioni di questo tipo sono:
 - Sistemi di raccomandazione tra gli utenti
 - Knowledge graph completion
 - Applicazioni di criminal intelligence



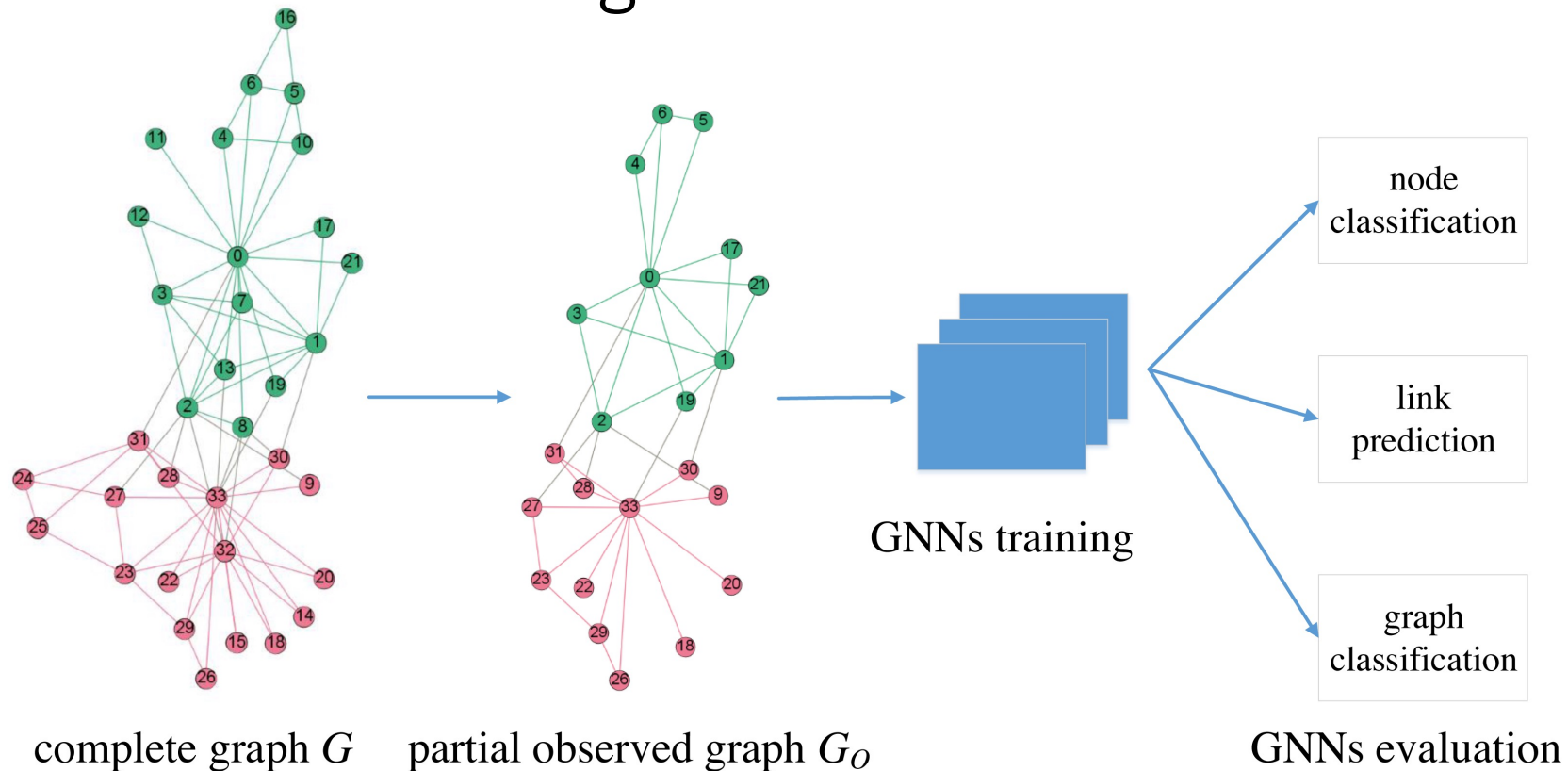
Task di analisi dei grafi

- **Graph Classification**

- Nella node classification ogni nodo viene trattato come un campione.
- Nella graph classification è l'insieme dell'intero grafo e delle sue connessioni che diventa un campione da testare
- Nella cheminformatica è il task più utilizzato poiché ogni molecola può essere rappresentata come un grafo per predire proprietà come solubilità, tossicità e la bioattività stessa



Task di analisi dei grafi

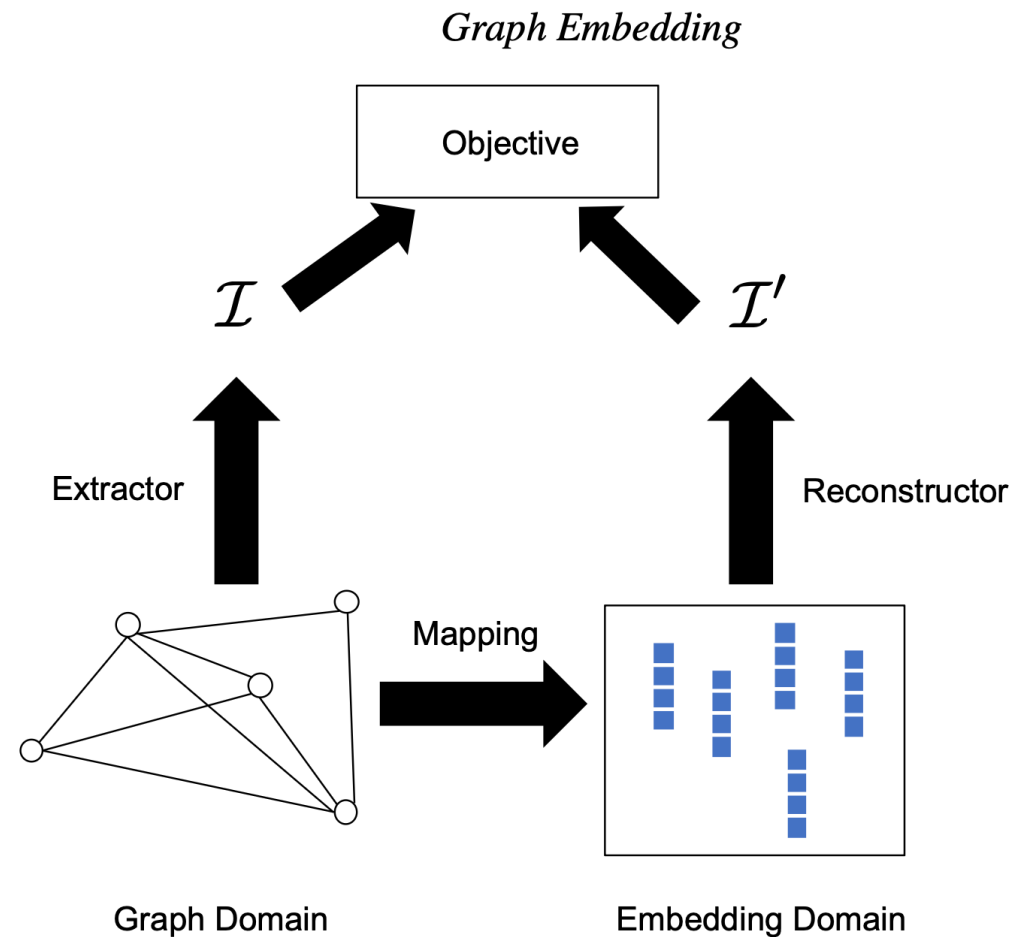


Processing

- I nodi possono essere rappresentati principalmente in due modi
 1. All'interno del grafo originale come elementi connessi dagli archi
 2. Come embedding in uno spazio vettoriale in cui ogni nodo viene rappresentato come un vettore di informazioni
- Ogni nodo può essere mappato nello spazio degli embedding
- Questa rappresentazione vettoriale può essere utilizzata come input per soddisfare un task in modo computazionalmente efficiente

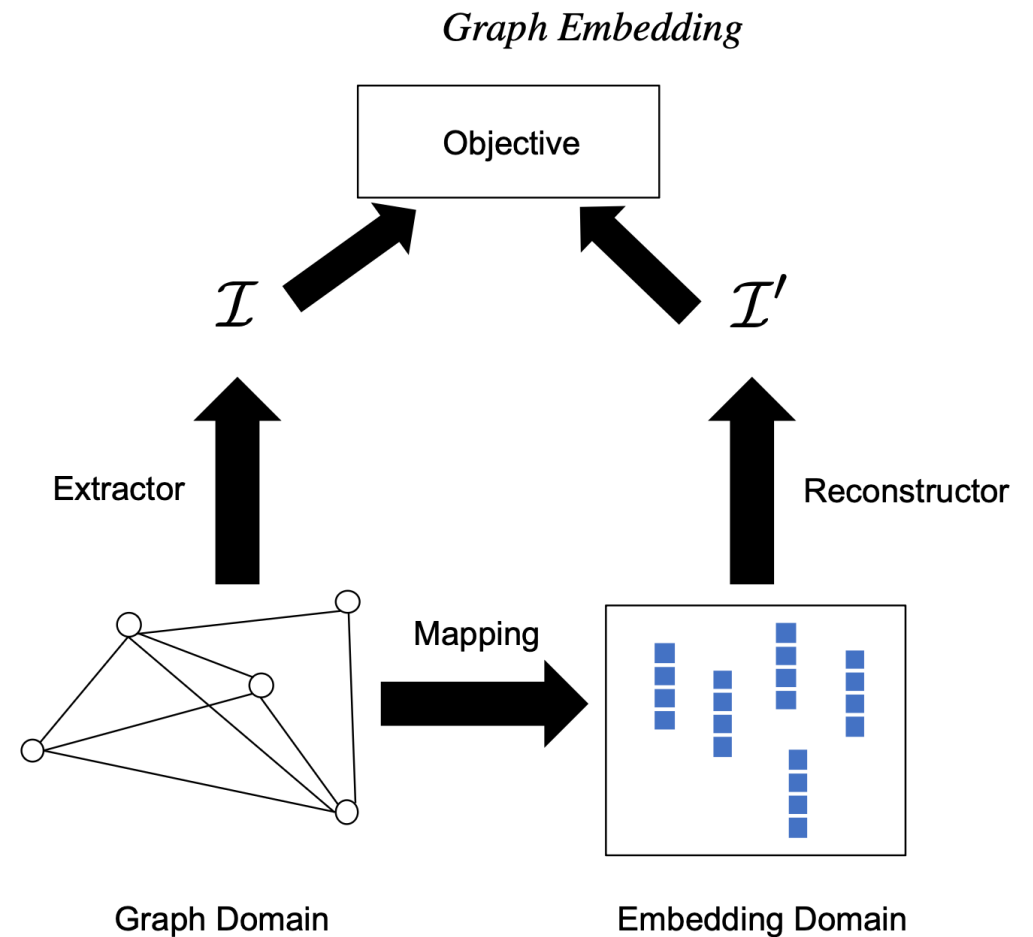
Uso degli embedding

- Il mapping deve avvenire preservando le feature rilevanti
- Deve avvenire minimizzando l'errore e soprattutto deve consentire una ricostruzione dell'informazione



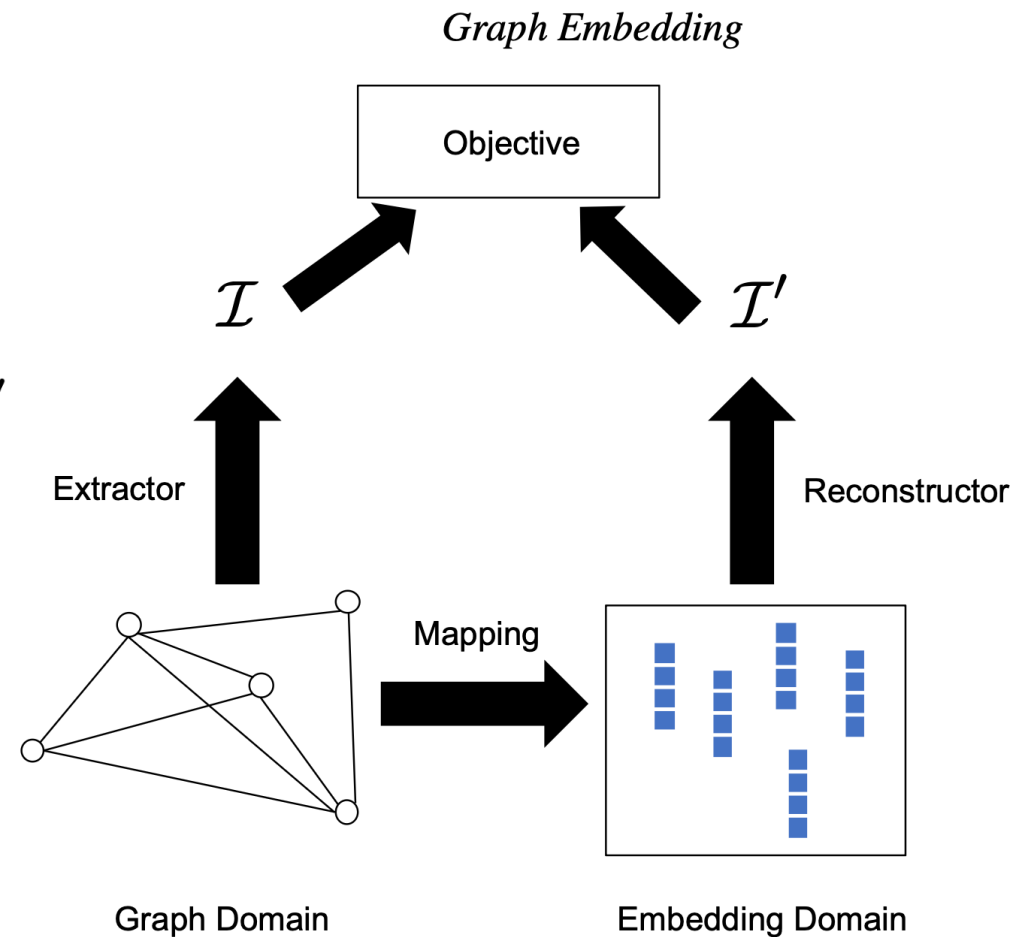
Uso degli embedding

1. Una funzione di mapping che mappa le informazioni nel dominio degli embedding
2. Un estrattore di informazioni atto ad ottenere le informazioni $\mathcal{I}$



Uso degli embedding

1. Un algoritmo che ricostruire le informazioni $\mathcal{I}'$ dall'embedding in modo coerente con le informazioni $\mathcal{I}$
2. Una funzione obiettivo che apprende i parametri per minimizzare le operazioni di mapping e ricostruzione



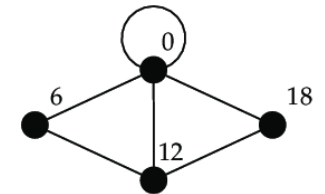
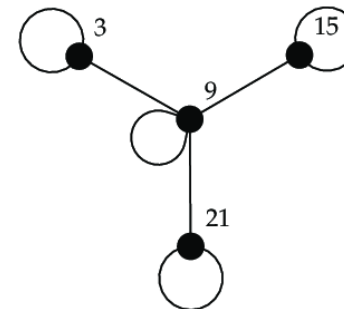
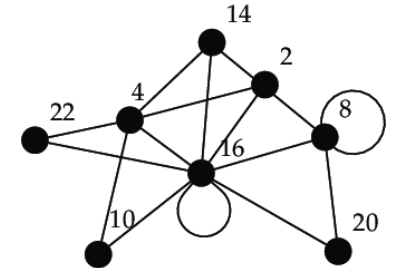
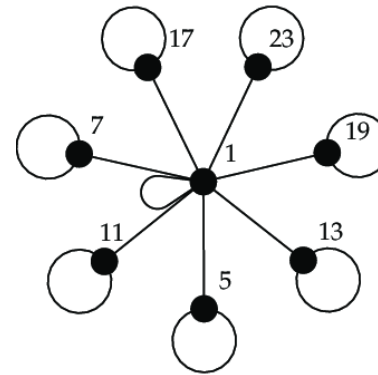
Pre-processing

- Il pre-processing sui grafi può essere eseguito in vari modi. Faremo riferimento al modulo `torch_geometric` per i metodi da utilizzare.
- **Ingestione, pulizia, indicizzazione**
 - Obiettivi: mappare gli ID dei nodi in indici $0 \dots N-1$, ripulire gli archi, garantire coerenza del grafo
 - Simmetrizzazione: `to_undirected` → Per ogni arco (u,v) viene aggiunto anche (v,u)
 - Per task non direzionali
 - Pro: semplice, coerente con modelli che assumono adiacenza non direzionale
 - Contro: raddoppia il numero di archi: da evitare se la direzione ha significato
 - Deduplica archi: `coalesce` → Unifica archi duplicati, aggregando i pesi con una riduzione (es. media)

Pre-processing

- Ingestione, pulizia, indicizzazione

- Self-loops: `add_self_loops`
 - Aggiungi (v, v) per ogni nodo: molti layer (es. GCN) beneficiano del messaggio identitario.



Pre-processing

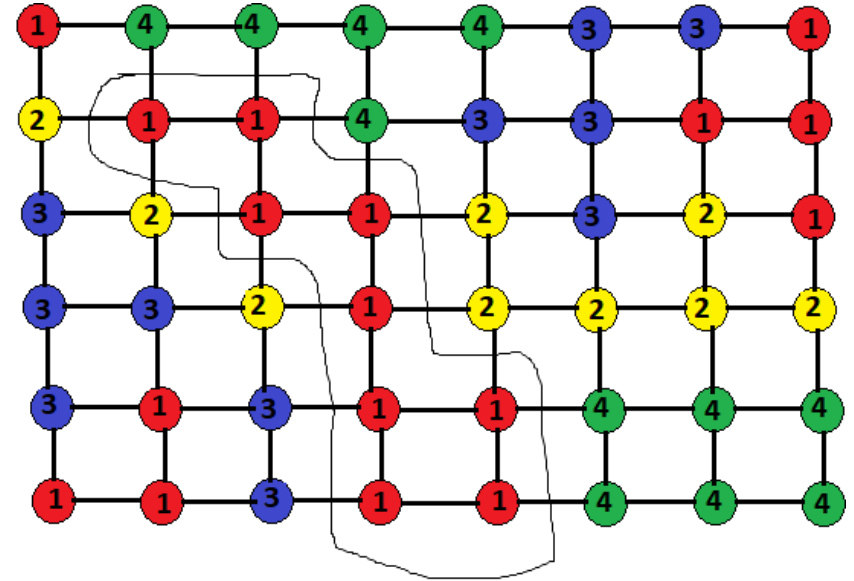
- Ingestione, pulizia, indicizzazione

- **Largest Connected Component (LCC)**

- Mantieni solo la componente connessa più grande se i nodi isolati non sono rilevanti per il task.

- Utilizziamo l'import:

```
from torch_geometric.utils import  
connected_components
```



Pre-processing

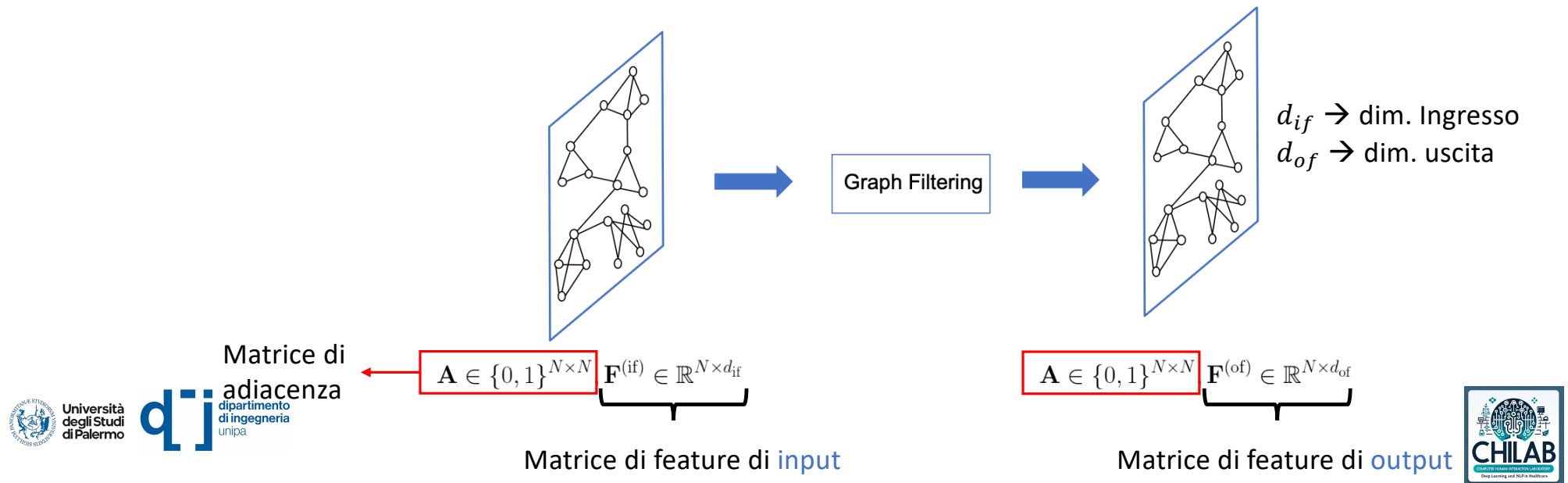
- **Numerical Processing**

- Normalizzazione della matrice di adiacenza
- Scaling numerico `StandardScaler` di `scikit-learn`
- Encoding dati categorici
- Gestione valori mancanti con mascheratura
 - La maschera casuale di un'aliquota dei che vengono azzerati aiuta il modello a discriminare tra valori mancanti e i valori realmente 0
- Edge features
 - Scalatura e normalizzazione dei pesi degli archi

Graph Neural Networks

- Le Graph Neural Network (GNN) sono una tipologia di reti neurali deputate all'analisi di informazioni strutturate a grafo

Graph Neural Networks



Graph Neural Networks

- La struttura di una Graph Neural Network (GNN) tipicamente prevede:

- **Graph Filtering** → *Node-focused task*

- **Graph Pooling**

} *Graph-focused task*

Graph Neural Networks

- La struttura di una Graph Neural Network (GNN) tipicamente prevede:

- **Graph Filtering** → ***Node-focused task***

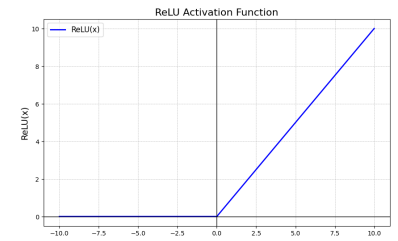
- **Graph Pooling**

Graph-focused task

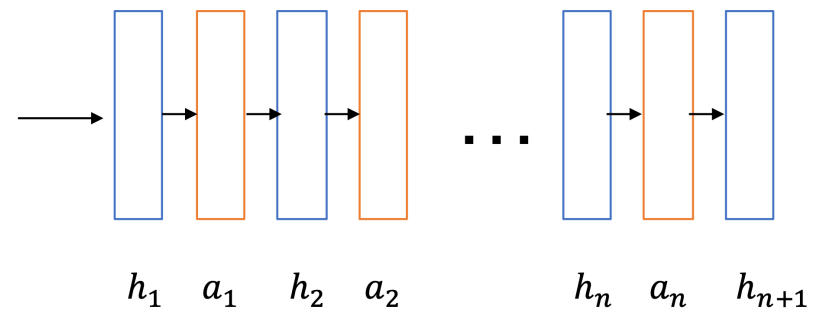
Node-focused task

- Quando la complessità è elevata il framework per questi task è tipicamente composto da:

- Graph Filtering
- Funzioni di attivazione non lineari



 Filtering Layer  Activation



Node-focused task

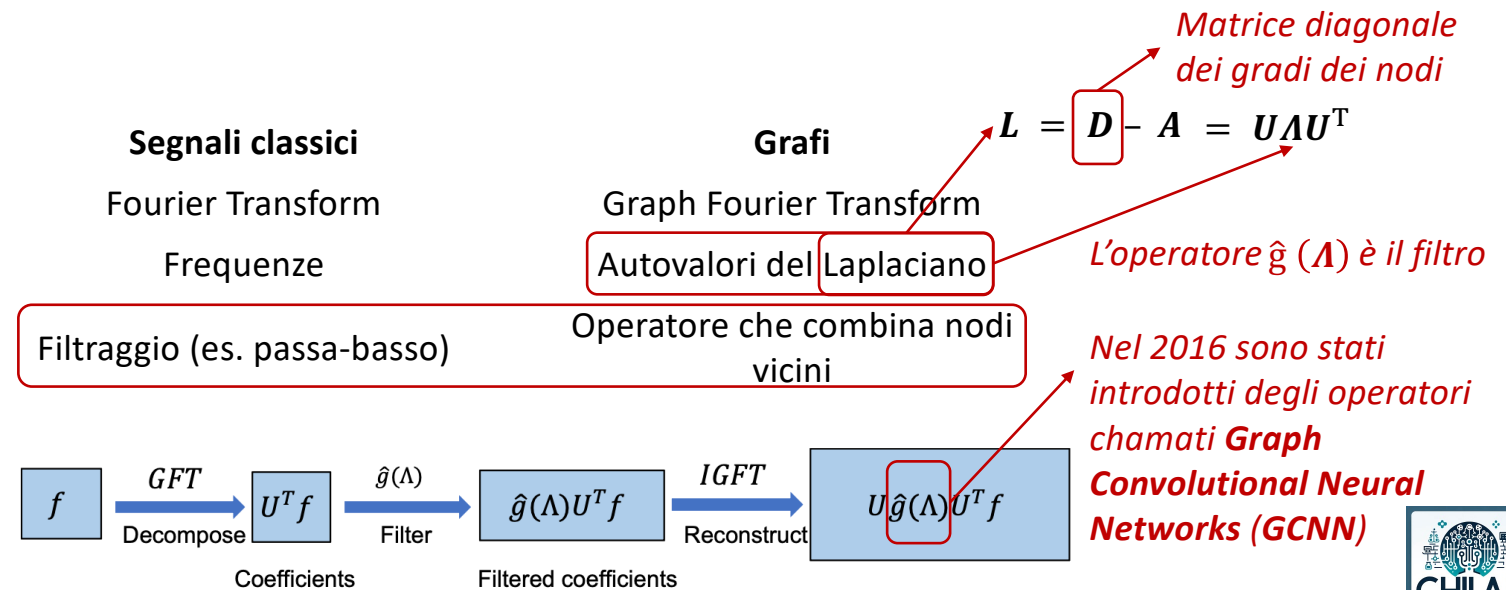
- I layer di Graph Filtering si possono raggruppare in due grandi categorie:
 1. Spectral-based graph filtering → utilizzano la teoria dei grafi spettrali per progettare l'operazione di filtraggio nel dominio spettrale.
 2. Spatial-based graph filtering → sfruttano esplicitamente la struttura del grafo (ovvero le connessioni tra i nodi) per eseguire il processo di affinamento delle feature nel dominio del grafo.

Spectral-based graph filtering

- L'idea chiave è che il grafo può essere visto come un *segnale discreto* definito su una struttura non euclidea:
 - Ogni nodo ha un vettore di feature x_i .
 - Tutto il grafo è un *segnale* X definito sui nodi.

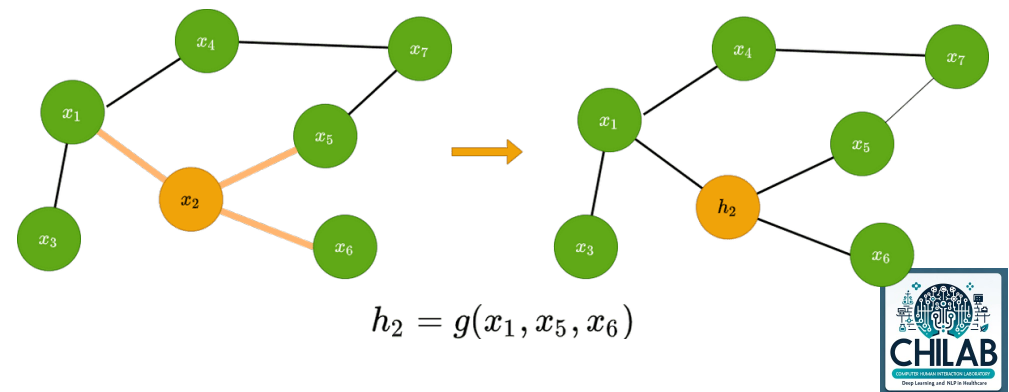
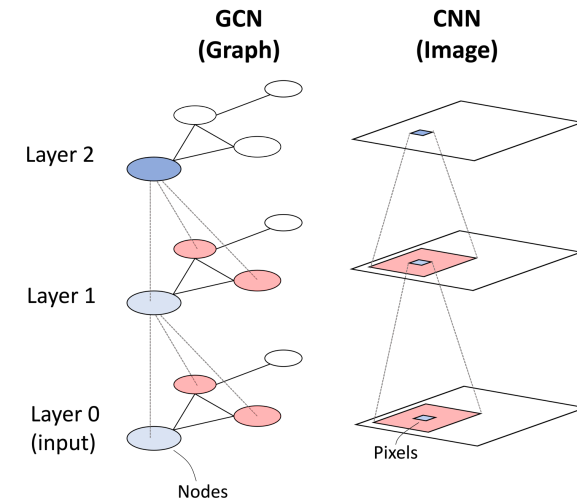
Spectral-based graph filtering

- In questo contesto, il **Graph Spectral Filtering** è l'analogo del filtraggio nel dominio della frequenza che si fa con la trasformata di Fourier per i segnali classici, ma applicato ai grafi.



Spectral-based graph filtering

- **GCNN**
- Esegue un'aggregazione di informazioni spaziali che coinvolge i vicini uno per volta
- Rende i nodi vicini più simili
- Questi filtri sono stati proposti ancor prima che il deep learning diventasse popolare



Spatial-based graph filtering

- Nel dominio spaziale, un **filtro** (o convoluzione) su un grafo significa:
 1. Considerare un nodo
 2. guardare i suoi vicini
 3. combinare le loro feature secondo una certa regola di aggregazione

$$h_i^{(l+1)} = \text{UPDATE}\left(h_i^{(l)}, \text{AGGREGATE}\{h_j^{(l)} : j \in \mathcal{N}(i)\}\right)$$

- Filtri orientati a sottografi locali

$h_i^{(l+1)}$ → rappresentazione del nodo i al layer l

$\mathcal{N}(i)$ → insieme dei nodi vicini di i

AGGREGATE → media, somma, max, attenzione

UPDATE → funzione (spesso un MLP o una combinazione lineare)

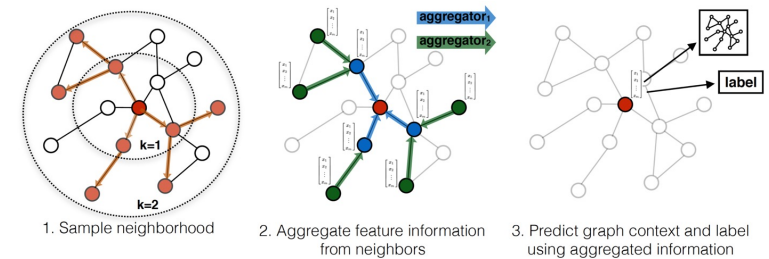
Spatial-based graph filtering

- Nel dominio spaziale, un **graph filter** definisce **quanto e come** un nodo ascolta i suoi vicini
 - Per esempio un filtro di ordine 1, ascolta solo i vicini diretti
 - Un **filtro di ordine 2** includerebbe anche i vicini dei vicini, e così via

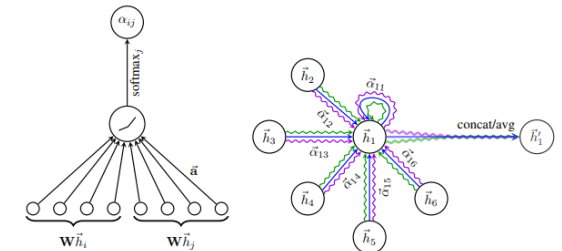
$$h_i^{(l+1)} = \sigma \left(W_1 h_i^{(l)} + W_2 \sum_{j \in \mathcal{N}(i)} h_j^{(l)} \right)$$

Spatial-based graph filtering

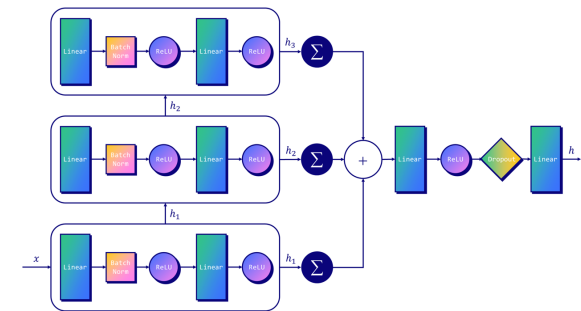
- A questa categoria appartengono:
- GraphSAGE (Hamilton et al., 2017)



- Graph Attention Network (GAT, Veličković et al., 2018)



- Graph Isomorphism Network (GIN, Xu et al., 2019)



Spatial-based graph filtering

- **GraphSAGE**

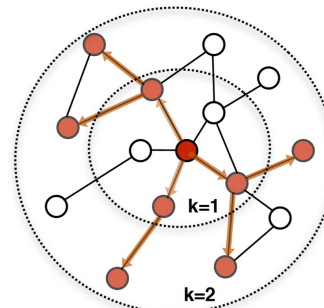
- Per un singolo nodo v_i generare nuove feature come:

$$\mathcal{N}_S(v_i) = \text{SAMPLE}(\mathcal{N}(v_i), S)$$

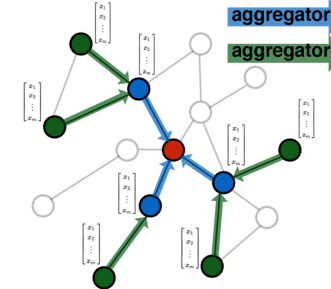
$$\mathbf{f}'_{\mathcal{N}_S(v_i)} = \text{AGGREGATE}(\{\mathbf{F}_j, \forall v_j \in \mathcal{N}_S(v_i)\})$$

$$\mathbf{F}'_i = \sigma([\mathbf{F}_i, \mathbf{f}'_{\mathcal{N}_S(v_i)}] \Theta)$$

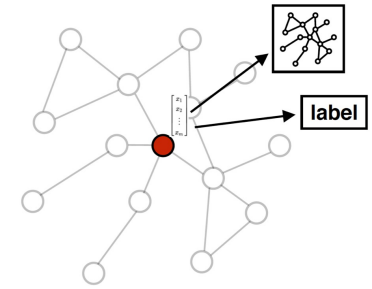
- **SAMPLE()** accetta un vicinato di nodi $\mathcal{N}_S(v_i)$ e campiona in modo casuale S elementi come output
- **AGGREGATE()** è una funzione che consente di combinare le informazioni provenienti dai nodi vicini.
- $\sigma([\mathbf{F}_i, \mathbf{f}'_{\mathcal{N}_S(v_i)}] \Theta)$ è la funzione che concatena le informazioni



1. Sample neighborhood



2. Aggregate feature information from neighbors

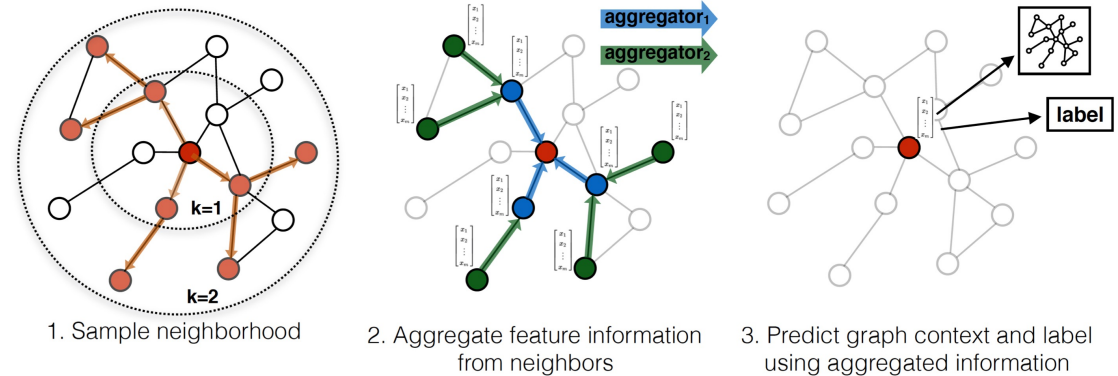


3. Predict graph context and label using aggregated information

Il filtro GraphSAGE è localizzato spazialmente poiché coinvolge solo i vicini a k salti, indipendentemente dall'aggregatore utilizzato.

Spatial-based graph filtering

- **GraphSAGE**



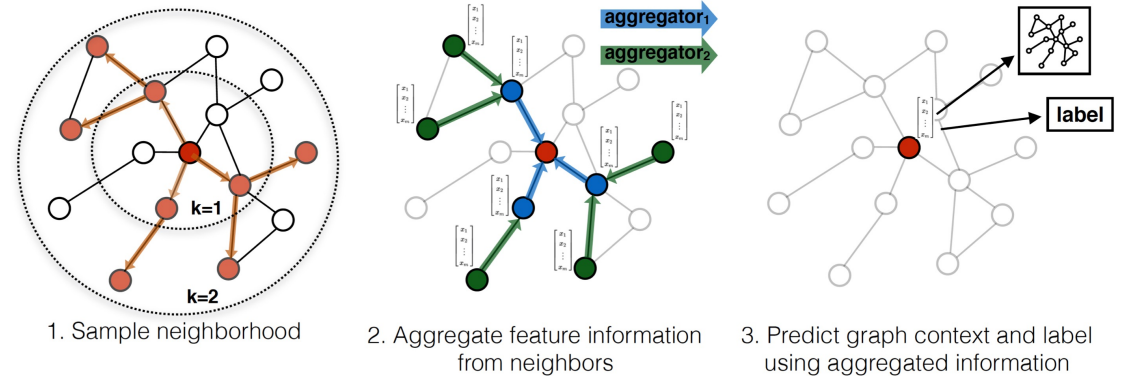
- La fase più rilevante è la **AGGREGATE()** che può essere applicata usando vari approcci:

1. Mean Aggregator

- Questo approccio si basa sul calcolo della media e conduce ad un risultato molto simile a quello ottenuto dalla GCN
- Quando si ha a che fare con un nodo v_i , entrambi prendono la media (ponderata) dei nodi vicini come sua nuova rappresentazione
- La differenza con la GCN sta nella gestione del dato di input che nel GraphSAGE viene concatenato con l'output della funzione di aggregazione nello step successivo, mentre nella GCN viene incluso nel processo di media ponderata

Spatial-based graph filtering

• GraphSAGE



- La fase più rilevante è la **AGGREGATE()** che può essere applicata usando vari approcci:

2. LSTM Aggregator

- L'aggregatore LSTM tratta l'insieme dei nodi vicini campionati $\mathcal{N}_S(v_i)$ del nodo v_i come una sequenza e utilizza l'architettura LSTM per elaborare la sequenza

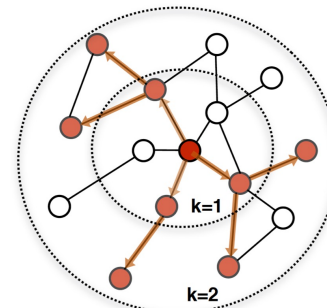
Spatial-based graph filtering

• GraphSAGE

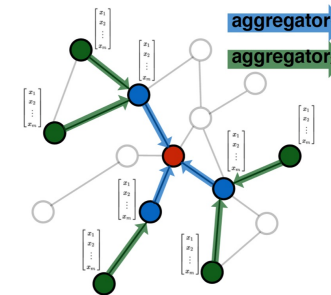
- La fase più rilevante è la **AGGREGATE()** che può essere applicata usando vari approcci:

3. Pooling Aggregator

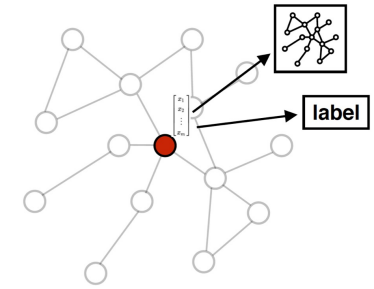
- L'operatore di pooling adotta l'operazione di pooling massimo per riassumere le informazioni provenienti dai nodi vicini
- Prima di riassumere i risultati, le feature di input in ciascun nodo vengono trasformate con un layer di una DNN



1. Sample neighborhood



2. Aggregate feature information from neighbors



3. Predict graph context and label using aggregated information

$$\mathbf{f}'_{\mathcal{N}_S(v_i)} = \underbrace{\max}_{\text{Pooling}}(\underbrace{\{\alpha(\mathbf{F}_j \mathbf{\Theta}_{\text{pool}})\}}_{\text{Matrice di trasformazione}}, \forall v_j \in \mathcal{N}_S(v_i)),$$

Non linear activation function

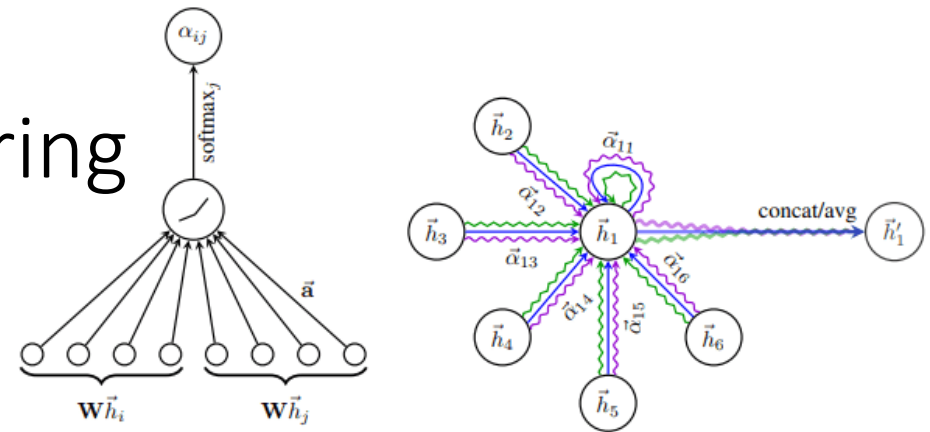
Spatial-based graph filtering

- **Graph Attention Network (GAT)**

- Il *GAT-filter* usa il meccanismo di Self-attention

- Questi filtri funzionano in modo simile alle GCN aggregando le informazioni dei nodi vicinali analizzati
- La differenza principale risiede che la procedura che aggrega le informazioni nel GAT punta a *differenziare* l'importanza dei vicini durante questo processo

- Questo processo avviene calcolando uno score di importanza per ognuno dei vicinali



Spatial-based graph filtering

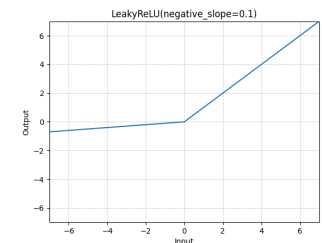
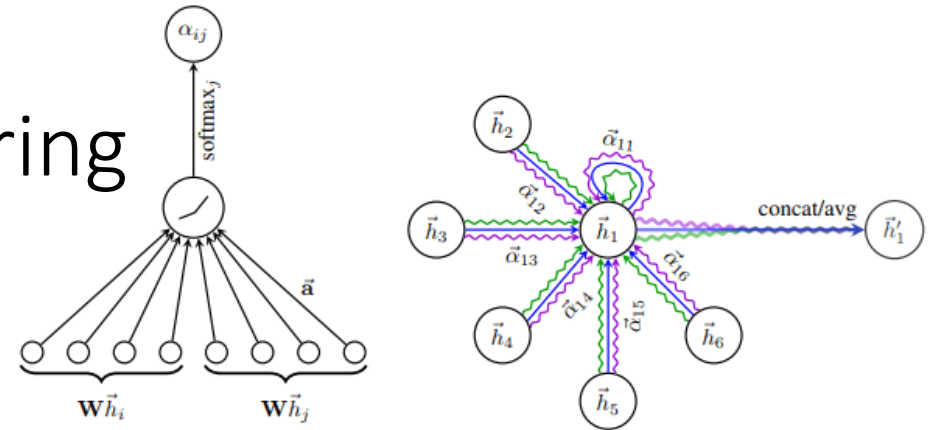
- **Graph Attention Network (GAT)**

- Lo score viene quindi calcolato come

$$e_{ij} = a(\mathbf{F}_i \Theta, \mathbf{F}_j \Theta)$$

- Θ è una matrice di parametri condivisi per fra i due nodi e a è una funzione di attenzione condivisa che è un layer di una rete feed forward

$$a(\mathbf{F}_i \Theta, \mathbf{F}_j \Theta) = \text{LeakyReLU}(\mathbf{a}^\top [\mathbf{F}_i \Theta, \mathbf{F}_j \Theta])$$



Spatial-based graph filter

Graph Attention Network (GAT)

- Lo score $e_{ij} = a(\mathbf{F}_i \Theta, \mathbf{F}_j \Theta)$ viene poi normalizzato attraverso una funzione softmax

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{v_k \in \mathcal{N}(v_i) \cup \{v_i\}} \exp(e_{ik})}$$

dove α_{ij} è lo score normalizzato dei nodi v_i e v_j

- In questo modo la nuova rappresentazione viene calcolata come segue

$$\mathbf{F}'_i = \sum_{v_j \in \mathcal{N}(v_i) \cup \{v_i\}} \alpha_{ij} \mathbf{F}_j \Theta$$

Spatial-based graph filter

Graph Attention Network (GAT)

- Lo score $e_{ij} = a(\mathbf{F}_i \Theta, \mathbf{F}_j \Theta)$ viene poi normalizzato attraverso una funzione softmax

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{v_k \in \mathcal{N}(v_i) \cup \{v_i\}} \exp(e_{ik})}$$

dove α_{ij} è lo score normalizzato dei nodi v_i e v_j

- Nel caso di una applicazione di un meccanismo multi-head vengono eseguiti M indipendenti meccanismi di attenzione i cui i risultati verranno poi concatenati come segue

Indica la concatenazione tra le teste

$$\mathbf{F}'_i = \parallel_{m=1}^M \sum_{v_j \in \mathcal{N}(v_i) \cup \{v_i\}} \alpha_{ij}^m \mathbf{F}_j \Theta^m$$

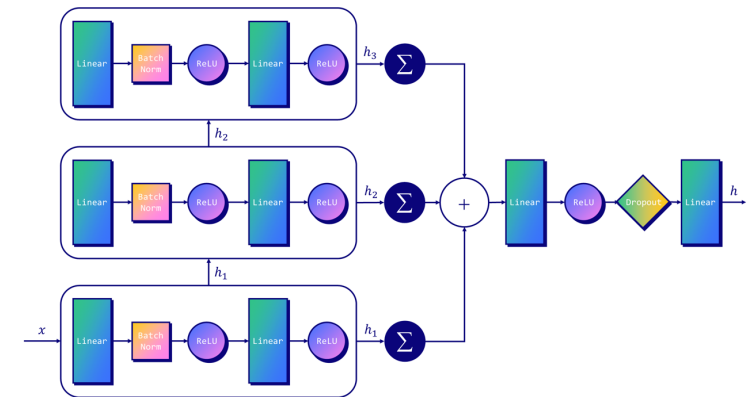
Spatial-based graph filter

Graph Isomorphism Network (GIN)

Sono tra le reti più importanti, perché rispondono a una domanda chiave nella teoria delle reti neurali sui grafi:

“Quanto è potente una GNN nel distinguere strutture diverse di grafi?”

- Ogni nodo **raccoglie i vettori** dei propri vicini
- Li **somma**, aggiungendo anche il proprio contributo
- Passa il risultato dentro un **MLP** che apprende una nuova rappresentazione



Spatial-based graph filter

Graph Isomorphism Network (GIN)

Sono tra le reti più importanti, perché rispondono a una domanda chiave nella teoria delle reti neurali sui grafi:

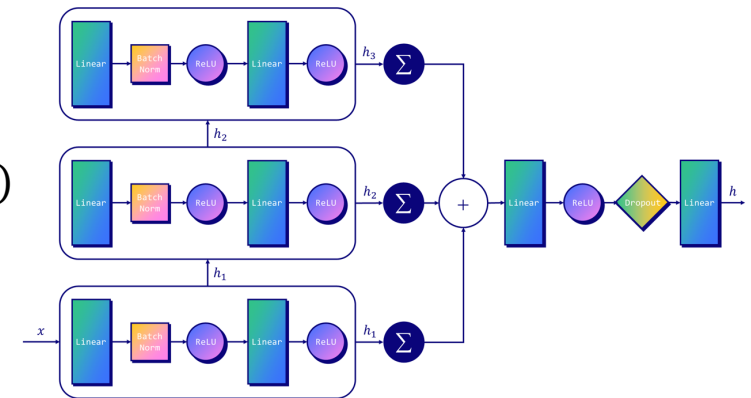
“Quanto è potente una GNN nel distinguere strutture diverse di grafi?”

La formula di aggiornamento del **GIN layer** è:

$$h_i^{(k)} = MLP^{(k)}((1 + \epsilon^{(k)}) \cdot h_i^{(k-1)} + \sum_{j \in \mathcal{N}(i)} h_j^{(k-1)})$$

dove:

- $h_i^{(k)}$: embedding del nodo i al layer k ,
- $\epsilon^{(k)}$: parametro (fisso o appreso) che controlla il peso del nodo stesso
- la **somma** aggrega le feature dei vicini
- un **MLP (Multi-Layer Perceptron)** non lineare combina e rimappa il risultato



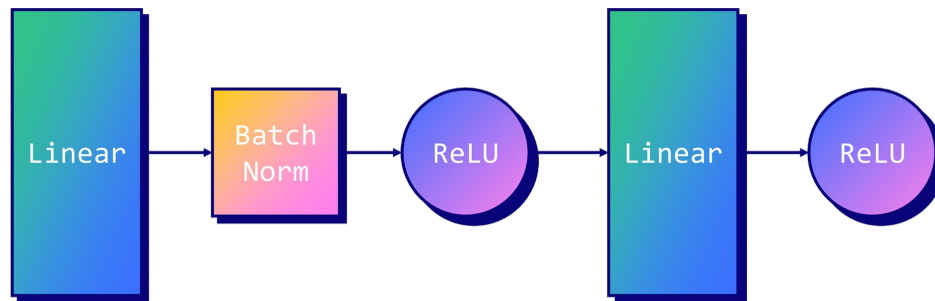
Spatial-based graph filter

Graph Isomorphism Network (GIN)

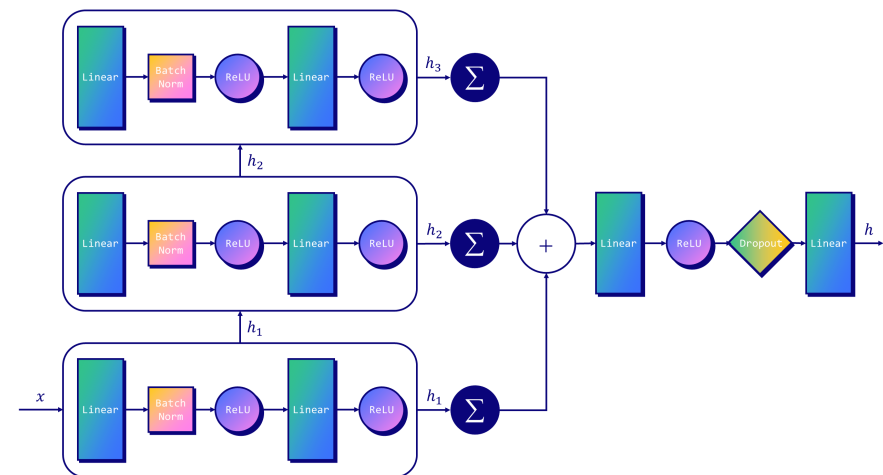
Sono tra le reti più importanti, perché rispondono a una domanda chiave nella teoria delle reti neurali sui grafi:

“Quanto è potente una GNN nel distinguere strutture diverse di grafi?”

GIN layer singolo



Architettura



Differenze tra i vari filtri

Filtri	Aggregazione	Concatenazione	Osservazioni
GCN	media pesata	lineare + ReLU	filtro “passa-basso”
GraphSAGE	mean / pooling / LSTM	concatenazione + linear	flessibile ma limitato
GAT	media pesata con attenzione	lineare + ReLU	dinamico ma costoso
GIN	somma	MLP non lineare	massima espressività teorica



Università
degli Studi
di Palermo

dj dipartimento
di ingegneria
unipa



Message Passing Framework

Message Passing Neural Networks è un framework generale da utilizzare nelle GNN

Molti filtri lo utilizzano come casi speciali GCN, GraphSAGE, GAT

- Per un nodo v_i l'update dei filtri avviene secondo

$$\mathbf{m}_i = \sum_{v_j \in \mathcal{N}(v_i)} M(\mathbf{F}_i, \mathbf{F}_j, \mathbf{e}_{(v_i, v_j)})$$

$$\mathbf{F}'_i = U(\mathbf{F}_i, \mathbf{m}_i),$$

dove $M()$ è la funzione messaggio, $U()$ è la funzione di aggiornamento e $\mathbf{e}_{(v_i, v_j)}$ sono le feature degli archi qualora presenti

Genera il messaggio dai nodi vicini



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa

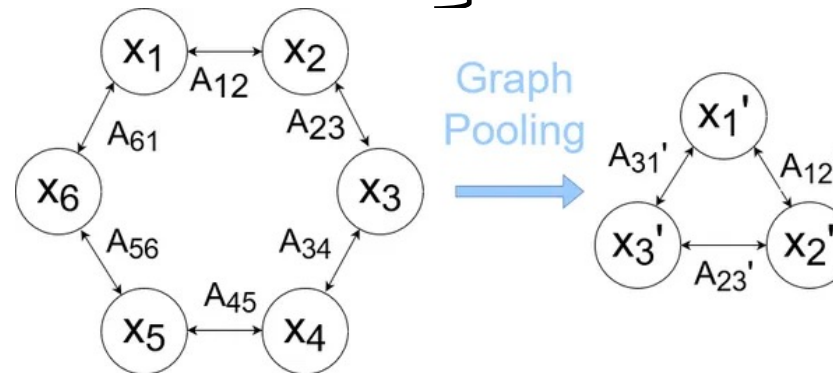


Graph-focused task

Negli approcci graph-focused oltre ai filtri troviamo il graph Pooling

- **Graph Filtering** → *Node-focused task*
- **Graph Pooling**

Graph-focused task



Graph-focused task

Graph Pooling

- Il pooling nei GNN serve a ottenere una rappresentazione compatta del grafo
- Mentre i filtri raffinano le feature dei nodi, il pooling aggrega tali informazioni per produrre rappresentazioni di livello grafo
- Generalmente il pooling si formula

$$\mathbf{A}^{(\text{op})}, \mathbf{F}^{(\text{op})} = \text{pool}(\mathbf{A}^{(\text{ip})}, \mathbf{F}^{(\text{ip})}).$$

- Può essere suddiviso in :
 - Flat Pooling
 - Hierarchical Pooling

Graph-focused task

Graph Pooling

Flat Pooling

- Il flat pooling genera direttamente una rappresentazione globale del grafo, ad esempio tramite media o massimo sui canali delle feature

$$\mathbf{f}_{\mathcal{G}} = \text{pool}(\mathbf{A}^{(\text{ip})}, \mathbf{F}^{(\text{ip})})$$

- $\mathbf{f}_{\mathcal{G}} \in \mathbb{R}^{1 \times d_{\text{op}}}$ è la rappresentazione del grafo
- È possibile introdurre meccanismi di attenzione per pesare l'importanza dei nodi, come nel gated global pooling

$$s_i = \frac{\exp(h(\mathbf{F}_i^{(\text{ip})}))}{\sum_{v_j \in \mathcal{V}} \exp(h(\mathbf{F}_j^{(\text{ip})}))}$$

h è una rete feedforward per mappare $F_i^{(\text{ip})}$ in uno scalare che verrà normalizzato con una *softmax*

Graph-focused task

Graph Pooling

Flat Pooling

Con lo score appreso la rappresentazione del grafo può essere descritta

$$\mathbf{f}_{\mathcal{G}} = \sum_{v_i \in \mathcal{V}} s_i \cdot \tanh(\mathbf{F}_i^{(\text{ip})} \Theta_{ip})$$

Dove Θ_{ip} sono i parametri a dover essere appresi mentre la funzione di attivazione *tanh* rimappa l'uscita $[-1; 1]$

Può essere sostituita con una funziona *identity* mantenendo coerente l'uscita

Graph-focused task

Graph Pooling

Hierarchical Pooling

Il pooling gerarchico conserva l'informazione strutturale del grafo riducendolo progressivamente. Due principali strategie sono adottate:

- **Downsampling-based pooling**
 - seleziona i nodi più rilevanti in base a uno score di importanza
- **Supernode-based pooling**
 - si apprende una matrice di assegnazione soft

Graph-focused task

Graph Pooling

Hierarchical Pooling

- **Downsampling-based pooling**

- seleziona i nodi più rilevanti in base a uno score di importanza

$$\mathbf{y} = \frac{\mathbf{F}^{(\text{ip})} \mathbf{p}}{\|\mathbf{p}\|},$$

$$\mathbf{F}^{(\text{ip})} \in \mathbb{R}^{N_{\text{ip}} \times d_{\text{ip}}}$$

E' la matrice che descrive le node feature

$$\mathbf{p} \in \mathbb{R}^{d_{\text{ip}}}$$

Dopo aver ottenuto $\mathbf{y}$ gli score vengono ordinati

$$\text{idx} = \text{rank}(\mathbf{y}, N_{\text{op}})$$

Dove N_{op} indica i nodi che devono essere ridotti

La posizione dei nodi rilevanti viene analizzata come segue

$$\mathbf{A}^{(\text{op})} = \mathbf{A}^{(\text{ip})}(\text{idx}, \text{idx}),$$

Estraendo righe e colonne attraverso l'indice idx per ognuno dei nodi

è un vettore da apprendere per proiettare le caratteristiche di input in punteggi di importanza

Graph-focused task

Graph Pooling

Hierarchical Pooling

- **Downsampling-based pooling**

$$\tilde{\mathbf{y}} = \sigma(\mathbf{y}(\text{idx}))$$

$$\tilde{\mathbf{F}} = \mathbf{F}^{(\text{ip})}(\text{idx}, :)$$

$$\mathbf{F}_p = \tilde{\mathbf{F}} \odot (\tilde{\mathbf{y}} \mathbf{1}_{d_{\text{ip}}}^\top),$$

Ottenuto l'indice dove $\sigma()$ è la funzione sigmoide che mappa il punteggio di importanza su (0,1) e $\mathbf{1}_{d_{\text{ip}}} \in R^{d_{\text{ip}}}$

È un vettore unico

Si noti che $\mathbf{y}(\text{idx})$ estrae gli elementi corrispondenti da $\mathbf{y}$ in base agli indici in idx e $\mathbf{F}^{(\text{ip})}(\text{idx})$ recupera le righe corrispondenti a idx

Graph-focused task

Graph Pooling

Hierarchical Pooling

- **Supernode-based pooling**

Invece di scartare nodi (come nel *downsampling*), questi metodi **raggruppano i nodi in cluster**, ognuno dei quali diventa un **supernodo**.

Il grafo ridotto conserva:

- le **feature aggregate** dei nodi originali
- la **struttura coarsenata** tra i cluster

Questa procedura avviene secondo diversi step:

Graph-focused task

Graph Pooling

Hierarchical Pooling

Questa procedura avviene secondo diversi step:

- **Matrice di assegnazione**

Ogni nodo v_i dell'input viene assegnato (hard o soft) a un **supernodo**:

$$S \in \mathbb{R}^{N_{in} \times N_{out}}$$

N_{in} : numero di nodi originali

N_{out} : numero di supernodi (ridotto)

S_{ij} : probabilità (soft assignment) o valore binario (hard assignment) che il nodo i appartenga al supernodo j .

- **Struttura del grafo ridotto**

La nuova matrice di adiacenza (tra supernodi) è:

$$A^{(op)} = S^T A^{(ip)} S$$

$A^{(ip)}$: matrice di adiacenza originale

$A^{(op)}$: matrice di adiacenza tra supernodi

Graph-focused task

Graph Pooling

Hierarchical Pooling

Questa procedura avviene secondo diversi step:

- Le feature aggregate dei supernodi si ottengono come:

$$F^{(op)} = S^T F^{(ip)}$$

$F^{(ip)} \in \mathbb{R}^{N_{in} \times d}$: feature dei nodi originali

$F^{(op)} \in \mathbb{R}^{N_{out} \times d}$: feature dei supernodi

In **DiffPool**, la matrice S è appresa tramite una **GCN**:

$$S = \text{softmax}\left(\text{GCN-Filter}\left(A^{(ip)}, F^{(ip)}\right)\right)$$

Poi:

$$A^{(op)} = S^T A^{(ip)} S$$

$$F^{(op)} = S^T F^{(inter)}$$

dove $F^{(inter)}$ è una trasformazione intermedia delle feature (es. tramite GCN).